mosaickit: Composing Two-Dimensional Diagrams

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-03 · v0.5.1 ·

Contents

1 Introduction **2**
1.1 Design 2
1.2 Mathematics and proofs 2
1.3 Reading guide 2
2 Installation **3**
2.1 Requirements 3
2.2 Installing the package 3
2.3 Development setup 3
3 Quick start **3**
4 Canvases and scenes **4**
4.1 Canvas specifications 4
4.2 Canvas 5
4.3 Scenes 5
4.4 Errors and warnings 6
5 Layers and axes **6**
5.1 Common fields 6
5.2 Primitive layers 6
5.3 Axes 8
6 Axis annotations **8**
6.1 Marks, notes and braces 8
6.2 Gutter columns 9
6.3 Spreading text along an axis 9
6.4 Lanes for braces 10
7 Layout geometry **11**
7.1 Rectangles 11
7.2 Orientation and segments 11
7.3 Polygons 12
7.4 The visual centre of a region 12
7.5 Brace outlines 13
8 Region and point labels **14**
8.1 Region labels 14
8.2 Point labels 14
8.3 Hard constraints 15
8.4 Callouts 16
9 Styles and colors **17**
9.1 Sparse styles 17
9.2 Style types 17
9.3 Colors 19
9.4 Palettes 19
10 Themes and configuration **20**
10.1 Style bundles and themes 20
10.2 Role resolution 20
10.3 Registering domain roles 21
10.4 Configuration 21
11 Parameters, grids and animation **22**
11.1 Parameters and expressions 22
11.2 Grids 23
11.3 Animation 24
12 Rendering **25**
12.1 Renderers 25
12.2 The Matplotlib renderer 25
12.3 Results and saving 25
12.4 Caching 26
A Proofs **26**
A.1 Packing and lanes 26
A.2 Rectangles, segments and polygons 27
A.3 The pole of inaccessibility 29
A.4 Labels 30
A.5 Styles, themes and binding 30
Change history **32**
Command Index **33**
References **34**

[*]Department of International Business, National Chengchi University, Taipei, Taiwan.
[†]Corresponding author. Email: contact@econ-viz.org

1 Introduction

The mosaickit package assembles two-dimensional diagrams from small, immutable pieces. A scene is an ordered list of layers (paths, filled regions, markers, text, arrows, labels, braces). Each layer names a semantic role, a theme maps roles to styles, and a renderer produces PNG, SVG, PDF, GIF or MP4. The package is domain-neutral. Domain packages such as principle-viz and utility-viz define their own models and role names, then pass mosaickit the layers to draw. Curve construction and TikZ export belong to geometry packages such as bezierkit.

1.1 Design

Four ideas run through the package.

Immutable values Layers, scenes, styles, themes and specifications are frozen dataclasses. A `Canvas` is a fluent builder around an immutable `Scene`; `snapshot()`, `copy()` and `bind()` never change a scene another object holds.

Sparse styles Every style field may be `None`, meaning inherit. A layer's explicit style sits on top of canvas overrides, configuration overrides, the theme and the primitive defaults (Section 10).

Roles, not colors Layers name what they are (`"primary"`, `"axes.note"`, `"mypkg.boundary"`); themes decide how that looks, and colors are palette names resolved when a canvas renders (Section 9).

Placement that covers nothing Region labels, point labels, braces and axis annotations are placed after everything else is drawn, by pure geometry in display pixels, so that text touches no line, marker, region or other text (Section 7, Section 8).

1.2 Mathematics and proofs

Automatic placement uses computational geometry: orientation tests, the even-odd rule, distances to a polygon's boundary, a best-first search for the point deepest inside a region, and the least-squares packing that keeps axis text apart. The chapters state what each routine guarantees as numbered definitions, lemmas, propositions and theorems; the proofs are collected in Appendix A, so the chapters can be read for the API alone. The same treatment covers the algebra of styles and themes (sparse merging, role resolution) and parameter binding. The standard references are de Berg et al. (2008) for the geometry and Barlow et al. (1972) for the order-restricted least squares behind Theorem 2.

1.3 Reading guide

Table 1 Chapter guide

Topic	Contents	Section
Building	Canvases, specifications, scenes	Section 4
	Paths, fills, markers, text, axes	Section 5
	Axis marks, notes and braces	Section 6
Placement	Layout geometry	Section 7
	Region and point labels	Section 8
Appearance	Styles, colors, palettes	Section 9
	Themes, roles, configuration	Section 10

Topic	Contents	Section
Output	Parameters, grids, animation	Section 11
	Renderers, caches, saving	Section 12

Start with Section 3, Section 4 and Section 5. Every figure in this manual is mosaickit output, drawn by the canvas or grid it illustrates and saved as PDF at the printed size.

The scripts that draw the figures of this manual come as one zip, sorted by topic: mosaickit-examples.zip.

2 Installation

2.1 Requirements

mosaickit requires Python 3.10 or later, NumPy 1.24 or later and Matplotlib 3.6 or later (below 4). On Python 3.10, it also installs tomli to read TOML. GIF output uses Pillow, which Matplotlib already depends on. MP4 output needs `ffmpeg` on the `PATH`.

2.2 Installing the package

```
uv add mosaickit                # the library
uv add "mosaickit==0.5.1"       # the version this manual describes
```

With pip, use `python -m pip install mosaickit`. Importing `mosaickit` does not import Matplotlib: the built-in renderer is loaded by name the first time a canvas renders (Section 12).

2.3 Development setup

```
git clone https://github.com/EconViz/mosaickit.git
cd mosaickit
uv sync --locked
uv run pre-commit install
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv run mypy
uv run lint-imports
uv build
```

The lockfile pins every development dependency. The continuous-integration workflow runs the same commands on Python 3.10, 3.11, 3.12 and 3.13. It then installs the built wheel into a clean environment, checks that bezierkit is absent, and runs the test suite against the wheel. Import contracts enforce the module boundaries: scenes, styles, themes and parameters never import the rendering or canvas modules, and the core never imports a domain package.

3 Quick start

A diagram needs a canvas, its layers and a call to save it:

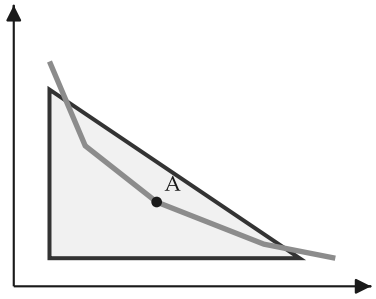


Figure 1
The quick-start diagram.

Example 1

```
from mosaickit import (
    Canvas, Fill, FillLayer, MarkerLayer, PathLayer, Stroke, TextLayer,
    quadrant_axes,
)

canvas = Canvas().extend(quadrant_axes(10, 10))
canvas.add(
    FillLayer(
        [(1, 1), (1, 7), (8, 1)],
        fill=Fill(color="#377EB8", opacity=0.12),
        z_index=-1,
    )
)
canvas.add(
    PathLayer(
        [(1, 8), (2, 5), (4, 3), (7, 1.5), (9, 1)],
        stroke=Stroke(color="#984EA3", width=2),
        id="curve",
    )
)
canvas.add(MarkerLayer([(4, 3)], id="point"))
canvas.add(TextLayer((4, 3), "A", offset=(6, 6)))
canvas.save("diagram.pdf")
```

`quadrant_axes(10, 10)` returns ordinary layers: two arrowed paths and their titles. The fill sits below everything else because `z_index=-1`. The path is the straight polyline through its points. mosaickit constructs no curves of its own, so a smooth curve must be supplied as a sequence of points or built and sampled by a geometry package. The text is offset 6 pt up and to the right of the point. Nothing is drawn until `save()`. That method renders the scene with the canvas's renderer (Matplotlib unless configured otherwise), writes the file and returns the paths it wrote.

The same canvas can be changed and saved again. `add()`, `extend()`, `remove()` and `clear()` return the canvas, so calls can be chained, while every earlier snapshot remains unchanged (Section 4). Labels that must not cover anything are also layers: replace the `TextLayer` with `PointLabelLayer((4, 3), "A")` and the renderer chooses the side (Section 8).

4 Canvases and scenes

4.1 Canvas specifications

`CanvasSpec` `CanvasSpec(x_range=(0, 10), y_range=(0, 10), width=6.0, height=6.0, dpi=300, x_label="X", y_label="Y", title=None)`

The physical dimensions and coordinate ranges of a diagram. `x_range` and `y_range` are the data ranges shown; `width` and `height` are the figure size in inches; `dpi` is an integer in $[1, 1200]$. The properties `x_min`, `x_max`, `y_min`, `y_max` read the ranges, and

`replace(**changes)` returns a copy with some fields changed. Ranges must be finite with `lo < hi`, sizes positive and finite; anything else raises `ConfigurationError`.

`Interval` `Interval(⟨lo⟩, ⟨hi⟩)`

A finite increasing interval, `lo < hi`, shared by specifications and axes.

4.2 Canvas

`Canvas` `Canvas(spec=None, theme=None, config=None, renderer=None, *, role_overrides=None)`

A fluent builder around an immutable scene. Arguments left `None` come from the configuration active when the canvas is created (Section 10.4): its `canvas_spec`, `theme` and `renderer`. `role_overrides` maps role names to `StyleBundles` applied on top of the theme and the configuration (Theorem 30).

`add(layer)`, `extend(layers)` Append one layer or several; return the canvas.

`remove(layer_id)`, `clear()` Remove the layer with that id (searching inside groups) or every layer; return the canvas. An unknown id raises `ConfigurationError`.

`snapshot()` The current `Scene`. Later calls to `add()` do not change it.

`copy()` A new canvas with the same specification, theme, configuration, renderer, overrides, scene and bindings.

`bind(parameter, value)`, `bind(mapping)` A new canvas whose scene has those parameters replaced by values (Section 11); the original keeps its parameters.

`render(*, renderer=None, cache=None)` Render the scene and return the renderer's result (Section 12).

`save(target, *, renderer=None, cache=None, **options)` Render, write `.png`, `.pdf` or `.svg`, close the result and return the written paths. The options are those of `SaveOptions`.

The builder methods change the canvas, never an existing scene. Each call replaces the canvas's scene with a new one, so an earlier snapshot, copy or bound canvas remains unchanged.

```python
from mosaickit import Canvas, PathLayer

canvas = Canvas()
before = canvas.snapshot()
canvas.add(PathLayer([(0, 0), (1, 1)], id="line"))
assert before.layers == ()              # the old snapshot is unchanged
assert canvas.snapshot().layers[0].id == "line"
```

4.3 Scenes

`Scene` `Scene(layers=(), metadata={})`

A persistent, immutable, ordered collection of layers. `add()`, `extend()`, `remove()` and `clear()` return new scenes; `Scene.empty()` is the empty one. Layer ids must be unique across the whole scene, groups included; a duplicate raises `ConfigurationError`. `ordered_layers` sorts the top-level layers by `z_index`, keeping insertion order among equals, which is the order in which they are drawn.

4.4 Errors and warnings

Table 2
Exceptions and warnings

Class	Raised or emitted when
`MosaicKitError`	Base class of the three errors below
`ConfigurationError`	A model, style, theme, specification or configuration value is invalid
`BindingError`	A parameter is missing, has the wrong type, or an expression cannot be evaluated
`RenderError`	A renderer cannot do what was asked: an unknown format or renderer, a missing legend entry
`LayoutWarning`	Automatic placement could not avoid every obstacle (Section 8)
`CacheBypassWarning`	A layer's model is not hashable, so it is rendered without caching

5 Layers and axes

5.1 Common fields

`Layer` `Layer(*, id=`⟨*uuid*⟩`, role="primary", z_index=0, visible=True, legend=None, model=None)`

The immutable base of every layer. All layers take these keyword-only fields after their own positional ones.

`id` A non-empty string, unique within the scene; a random hex string by default. Legends, region labels and `remove()` refer to layers by id.

`role` A dotted role name such as `"primary"` or `"mypkg.boundary"`; it selects the layer's style from the theme (Section 10). Each layer type has its own default and a fallback category consulted last.

`z_index` A finite drawing order; higher is drawn later, on top. A group adds its own `z_index` to its children's.

`visible` `False` leaves the layer out of the render.

`legend` The legend label of the layer; layers without one are not in legends.

`model` Any domain object the layer was built from. It takes part in render caching (Section 12) but not in equality or binding.

Coordinates are pairs of finite real numbers in data units, or parameter expressions (Section 11). Invalid geometry raises `ConfigurationError` when the layer is created, not when it is drawn.

5.2 Primitive layers

Table 3
Primitive layers

Layer	Default role	Draws
`PathLayer(path, stroke)`	`primary`	The polyline through at least two points

Layer	Default role	Draws
`FillLayer(boundary, fill, stroke)`	`region`	The polygon on at least three points, filled and outlined
`MarkerLayer(points, marker)`	`point`	A marker at each of at least one point
`TextLayer(position, text)`	`text`	Text at a point
`ArrowLayer(start, end, stroke)`	`annotation`	A straight arrow, optionally labelled
`LegendLayer(entries, style)`	`legend`	A legend of labelled layers
`GroupLayer(children)`	`primary`	Nothing of its own; groups layers

`PathLayer` `PathLayer(path, stroke=None, arrow_placement=ArrowPlacement.END, clip=True, *, ...)`

Joins its points with straight segments. When the resolved stroke has an `arrow` style, arrowheads are drawn at the `START`, `END` or `BOTH` ends, aligned with the end segments. `clip=False` lets the line run past the plot edge at full width, which is what the axis presets use.

`FillLayer` `FillLayer(boundary, fill=None, stroke=None, *, ...)`

A closed polygon filled with `fill` and outlined with `stroke`. Its id is what a `RegionLabel Layer` names.

`MarkerLayer` `MarkerLayer(points, marker=None, *, ...)`

One marker per point. Markers are drawn whole. A marker centred on the plot edge, such as a point on an axis, is not cut in half; one centred outside the plot is omitted.

`TextLayer` `TextLayer(position, text, style=None, offset=(0, 0), anchor="center", math=False, *, ...)`

Text at `position`, moved by `offset` points. `anchor` names the point of the text box placed there: `center`, `left`, `right`, `top`, `bottom`, `top-left`, `top-right`, `bottom-left` or `bottom-right`. With `math=True` the text is set as mathematics (`"a_1"` gives a_1).

`ArrowLayer` `ArrowLayer(start, end, stroke=None, label=None, arrow_placement=ArrowPlacement.END, *, ...)`

A straight arrow from `start` to `end`, with an open head unless the stroke names another `ArrowStyle`; `label` is written at its midpoint. A dashed, dotted or dash-dot stroke dashes the shaft and leaves the heads solid. Arrowheads are not clipped to the plot.

`LegendLayer` `LegendLayer(entries=(), style=None, *, ...)`

A legend listing the layers whose ids are in `entries`, or every layer that has a `legend` label when `entries` is empty. An id that is missing or has no label raises `RenderError`. `Legend Style(visible=False)` hides it.

`GroupLayer` `GroupLayer(children, *, ...)`

A layer made of layers. Children are drawn as if they were in the scene, with the group's `z_index` added to theirs; an invisible group hides them all.

5.3 Axes

`AxisSpec` `AxisSpec(extent, arrow=None, label=None)`
`build_axes` `build_axes(x, y) -> list[Layer]`

`build_axes` turns two axis specifications into ordinary layers with the role `axes`. When neither has an `arrow`, the result is a closed frame around the extents; otherwise each axis is a line along $y = 0$ or $x = 0$, with filled triangle heads at the `ArrowPlacement` given. Titles sit past the arrow tips: the x title to the right, the y title above, 6 pt away.

`quadrant_axes` `quadrant_axes(x_max, y_max)`
`crosshair_axes` `crosshair_axes(x_range, y_range)`
`box_frame` `box_frame(total_x, total_y)`

Three presets, respectively: two axes from the origin with heads at the far ends; two axes through the origin with heads at both ends; and a frame without heads.

Every axis is made of `PathLayer` and `TextLayer`, so it can be restyled through the `axes` role, removed by id (`axes.x`, `axes.y`, `axes.frame`, `axes.x.label`, `axes.y.label`) or replaced by hand-made layers.

6 Axis annotations

The y axis is the plot's left edge and the x axis its bottom edge; the space outside them is the *gutter*. Three layer types place text in the gutter, and a fourth draws braces inside the plot.

6.1 Marks, notes and braces

`AxisMarkLayer` `AxisMarkLayer(axis, value, label, math=False, style=None, *, role="axes", ...)`

New: 0.3.0 A short symbol next to axis `"x"` or `"y"` at `value`, outside the plot, such as a_0 or p^*.

`AxisNoteLayer` `AxisNoteLayer(axis, value, text, style=None, *, role="axes.note", ...)`

New: 0.3.0 An explanation of `value`, possibly on several lines, in the outermost gutter column. The `axes.note` role makes notes smaller and lighter than marks in the default theme (9 pt, `grey-600`).

`BraceLayer` `BraceLayer(axis, start, end, label=None, side="inside", *, role="axes", math=False, style=None, stroke=None, ...)`

New: 0.3.0 A curly brace over `start`..`end` on an axis, with an optional label. `side="outside"` draws it in the gutter, past the marks; `"inside"` draws it just inside the plot and places its label so that it covers no line, point, region or text, on a leader when nothing past the tip is free.

`SpanBraceLayer` `SpanBraceLayer(start, end, label=None, side="below", *, role="axes", math=False, style=None, stroke=None, ...)`

New: 0.4.0 A brace between two points inside the plot. The span must be horizontal (`side` `"above"` or `"below"`) or vertical (`"left"` or `"right"`); the brace bulges to `side`, 4 pt clear of the points and 8 pt deep, and its label sits past the tip, placed so it covers nothing (Section 8).

```
from mosaickit import AxisMarkLayer, AxisNoteLayer, BraceLayer, Canvas,
quadrant_axes

canvas = Canvas().extend(quadrant_axes(10, 10))
for value, symbol, note in [(7, "a_1", "Upper\nvalue"), (5, "a_0",
"Lower\nvalue")]:
    canvas.add(AxisMarkLayer("y", value, symbol, math=True))
    canvas.add(AxisNoteLayer("y", value, note))
```

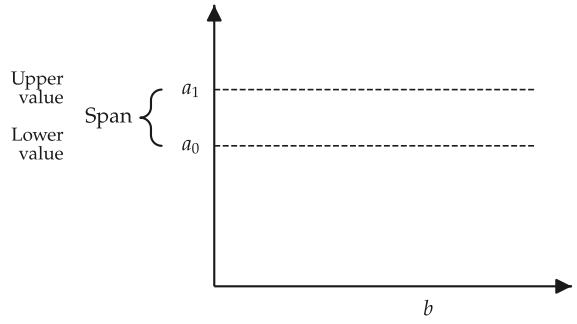


Figure 2
Marks, notes and an outside brace in the y gutter (guide lines added).

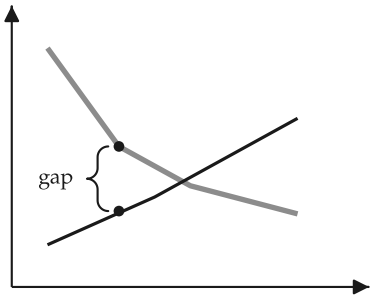


Figure 3
A span brace between two points, labelled to the left.

```
canvas.add(BraceLayer("y", 5, 7, "Span", side="outside"))
canvas.add(AxisMarkLayer("x", 6, "b", math=True))
```

6.2 Gutter columns

Columns run outward from the axis line, starting 5 pt away and separated by 8 pt: marks, then one column per lane of outside braces, then notes. Each column is as wide as its widest item. An empty column takes no space and adds no gap (Figure 2). The layout must also keep neighbouring texts within a column from overlapping and decide which braces may share a lane.

6.3 Spreading text along an axis

Each text in a column is an interval along the axis, centred on the value it labels. When intervals overlap they are moved apart, keeping their order and moving as little as possible in the least-squares sense.

Definition 1 (Order-preserving packing). Let $c_1, \dots, c_n$ be centres, $s_1, \dots, s_n \geq 0$ sizes and $g \geq 0$ a gap, numbered so that $c_1 \leq \cdots \leq c_n$ (ties in input order). A *packing* is a vector $x \in \mathbb{R}^n$ with

$$x_{k+1} - x_k \geq \frac{s_k + s_{k+1}}{2} + g, \quad k = 1, \dots, n-1,$$

and the *order-preserving packing problem* is to minimize $\sum_k (x_k - c_k)^2$ over all packings.

`spread` `mosaickit.layout.stack1d.spread(centers, sizes, gap=0.0) -> tuple[float, ...]`

New: 0.3.0

Solves the problem by merging runs of overlapping items into clusters, packing each cluster tightly around the mean of its members' targets, and merging again while a cluster



Figure 4
Targets (top) and their spread (bottom).

runs into the one before it. The result is returned in input order. Mismatched lengths or a negative size raise `ValueError`.

Theorem 2 (Spreading is optimal). `spread` *returns the unique solution of the order-preserving packing problem of Definition 1.*

The procedure is the pool-adjacent-violators algorithm (PAVA) from isotonic regression (Ayer et al., 1955): subtracting the packed offsets turns the gap constraints into $y_1 \le \cdots \le y_n$.

Corollary 3 (Properties of a spread). *Let x be the result of* `spread`*. Then (i) any two items $i \neq j$ satisfy $|x_i - x_j| \ge \frac{s_i+s_j}{2} + g$; (ii) if the centres already form a packing, $x = c$; (iii) within every cluster the displacements sum to zero, so the cluster's mean position equals its members' mean target.*

In Figure 4 the first three items overlap and form one cluster centred at their mean target. The other two are already clear and do not move.

6.4 Lanes for braces

Braces whose spans (with their labels) come too close must go in different lanes, that is, at different distances from the axis.

Definition 4 (Conflicting intervals). Intervals $[a, b]$ and $[a', b']$ *conflict* for a gap $g \ge 0$ when neither $b + g \le a'$ nor $b' + g \le a$. A *lane assignment* gives each interval a lane number so that no two conflicting intervals share a lane.

`assign_lanes` `mosaickit.layout.stack1d.assign_lanes(intervals, gap=0.0) -> tuple[int, ...]`

New: 0.3.0 First fit takes the intervals in input order and gives each the lowest lane in which it conflicts with nothing already there. Endpoints may be given in either order.

Theorem 5 (First fit uses the fewest lanes). `assign_lanes` *always returns a lane assignment. If the intervals are given in increasing order of their lower ends and each satisfies $b - a + g > 0$, it uses exactly ω lanes, where ω is the largest number of pairwise conflicting intervals; no lane assignment uses fewer.*

In other orders first fit can use more lanes than necessary, so add braces along an axis from low to high values when they may overlap.

`gutter_columns` `mosaickit.layout.gutter.gutter_columns(mark_width, brace_widths, note_width, *, start, gap)`

New: 0.3.0 The pure column layout behind the gutter: returns `GutterColumns` with `marks`, `braces` (one `Band` per lane, innermost first) and `notes`, each a `Band(near, far)` measured outward from the axis, and `extent`, the farthest edge.

7 Layout geometry

Label placement runs after everything else is drawn and uses display coordinates: pixels, with y increasing upward. The package `mosaickit.layout` contains this geometry and imports nothing from the renderers, so it can be tested and reused independently. The functions in this chapter are in `mosaickit.layout.geometry`; `Rect` and `polylabel` are also exported from `mosaickit.layout`. A point is a pair `(x, y)`, a segment is a pair of points, and a polygon is a sequence of points whose last point joins the first.

7.1 Rectangles

`Rect` `Rect(x0, y0, x1, y1)`

The closed axis-aligned rectangle $[x_0, x_1] \times [y_0, y_1]$. `Rect.centered(center, width, height)` builds one around a point. It has `width`, `height`, `center`, `inflate(pad)`, `corners()` (counter-clockwise from (x_0, y_0)), `edges()`, `contains(point)` (edges included), `within(other)`, `intersects(other)` (touching counts) and `nearest_point(point)`.

Lemma 6 (Nearest point of a rectangle). *For a rectangle R and a point p, `nearest_point` returns*

$$q = \big(\min(\max(p_x, x_0), x_1), \min\big(\max\big(p_y, y_0\big), y_1\big)\big).$$

This is the unique point of R closest to p.

Callout leaders end at this point, so a leader is as short as the label's position allows.

7.2 Orientation and segments

Definition 7 (Orientation). For points a, b, c in the plane,

$$\operatorname{orient}(a, b, c) = (b_x - a_x)\big(c_y - a_y\big) - \big(b_y - a_y\big)(c_x - a_x),$$

the cross product of $b - a$ and $c - a$.

Lemma 8 (Orientation test). $\operatorname{orient}(a, b, c)$ *is positive when c lies to the left of the directed line from a to b (the turn $a \to b \to c$ is counter-clockwise), negative when it lies to the right, and zero when a, b, c are collinear. It is invariant under cyclic permutation and changes sign when two arguments are swapped.*

`segments_intersect` `segments_intersect(a, b, c, d) -> bool`

Whether the closed segments $[a, b]$ and $[c, d]$ share a point. With $d_1 = \operatorname{orient}(c, d, a)$, $d_2 = \operatorname{orient}(c, d, b)$, $d_3 = \operatorname{orient}(a, b, c)$, $d_4 = \operatorname{orient}(a, b, c)$, it answers yes when $d_1 d_2 < 0$ and $d_3 d_4 < 0$, or when some d_i is zero and its point lies in the bounding box of the other segment (Cormen et al., 2009).

Theorem 9 (Segment intersection). *In exact arithmetic, `segments_intersect(a, b, c, d)` is true if and only if $[a, b] \cap [c, d] \neq \emptyset$.*

`rect_hits_segment` `rect_hits_segment(rect, a, b) -> bool`

Whether the segment touches the rectangle: an endpoint lies in it, or the segment meets one of its four edges.

Lemma 10 (Segments and rectangles). `rect_hits_segment(R, a, b)` *is true if and only if $[a, b] \cap R \neq \emptyset$.*

7.3 Polygons

Definition 11 (Simple polygon). A polygon $P = (p_0, \dots, p_{m-1})$, $m \geq 3$, is *simple* when its edges $[p_i, p_{i+1}]$ (indices mod m) meet only at shared endpoints of consecutive edges and its vertices are not all collinear. Its boundary ∂P is the union of the edges; by the Jordan curve theorem the complement of ∂P has one bounded component, the *interior* $\operatorname{int} P$, and one unbounded one, the *exterior*. We write $\overline{P} = \operatorname{int} P \cup \partial P$.

`polygon_edges` `polygon_edges(polygon) -> tuple[Segment, ...]`
`distance_to_boundary` `distance_to_boundary(point, polygon) -> float`

The edges, last vertex to first included, and the Euclidean distance from a point to the nearest edge. Each edge distance projects the point onto the edge's line and clamps the parameter to $[0, 1]$.

Lemma 12 (Distance to a segment). *For $a \neq b$, let*

$$t^* = \operatorname{clamp}\big((p - a) \cdot (b - a) / \, |b - a|^2, 0, 1\big).$$

Then $a + t^*(b - a)$ is the point of $[a, b]$ nearest to p.

`point_in_polygon` `point_in_polygon(point, polygon) -> bool`

The even-odd rule: follow the horizontal ray from the point to the right and count the edges it crosses. An edge counts when exactly one of its endpoints lies strictly above the ray's line and the crossing is right of the point (Haines, 1994).

Proposition 13 (Even-odd rule). *Let P be simple and $p \notin \partial P$. Then `point_in_polygon(p, P)` is true if and only if $p \in \operatorname{int} P$.*

`rect_inside_polygon` `rect_inside_polygon(rect, polygon) -> bool`
`rect_overlaps_polygon` `rect_overlaps_polygon(rect, polygon) -> bool`

Inside: all four corners pass the even-odd test and no edge of the polygon touches the rectangle. Overlaps: some corner passes, or some edge touches the rectangle (which also catches a polygon lying wholly inside it).

Theorem 14 (Rectangles in polygons). *Let P be simple and R a rectangle. Then `rect_inside_polygon(R, P)` is true if and only if $R \subset \operatorname{int} P$.*

Corollary 15 (Rectangles overlapping polygons). *Let P be simple and R a rectangle. Then `rect_overlaps_polygon(R, P)` is true if and only if $R \cap \overline{P} \neq \emptyset$.*

`ray_exit` `ray_exit(origin, direction, polygon) -> float`

The largest $t \geq 0$ at which the ray $o + tr$ meets an edge that is not parallel to it, or 0 if there is none. Callouts use it to start their search where the ray leaves the region.

Proposition 16 (Leaving a polygon for good). *Let P be simple, $r \neq 0$ and $T =$ `ray_exit(o, r, P)`. Then $o + tr \notin \overline{P}$ for every $t > T$.*

7.4 The visual centre of a region

The centroid of a region can lie outside it (Figure 5). A label belongs at the deepest interior point, which is farthest from the boundary.

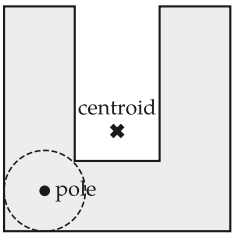


Figure 5
Pole, largest disc and centroid.

Definition 17 (Signed distance and pole). For a simple polygon P the *signed distance* is $f(p) = d(p, \partial P)$ for $p \in \overline{P}$ and $f(p) = -d(p, \partial P)$ otherwise. A *pole of inaccessibility* of P is a point where f attains its maximum f^*, the radius of the largest disc inside P.

Lemma 18 (Signed distance is 1-Lipschitz). *For all points p, q, $|f(p) - f(q)| \le |p - q|$.*

`polylabel` `polylabel(polygon, precision=1.0) -> Point`

A best-first search over square cells (Agafonkin, 2016). The bounding box is covered with squares as wide as its shorter side. Each cell with centre c and half-size h is given the bound $f(c) + h\sqrt{2}$ and kept in a priority queue, largest bound first. The best centre seen so far is remembered, starting with the centre of the bounding box; a popped cell whose bound exceeds the best value by more than `precision` is split into four, the others are dropped. A polygon with a zero-width or zero-height bounding box returns its first vertex.

Theorem 19 (Pole of inaccessibility). *Let P be simple, with a bounding box of positive width and height, and let $\varepsilon > 0$ be the precision. Then* `polylabel` *terminates and returns a point q with $f(q) \ge f^* - \varepsilon$. In particular q lies in* int P *whenever $f^* > \varepsilon$.*

For a triangle the pole is the incentre. Placement calls `polylabel` with the default precision of one pixel, finer than any text-placement increment.

7.5 Brace outlines

`brace_outline` `brace_outline(start, end, *, base, depth, direction, axis="y", samples=12)`
`Brace`

New: 0.3.0

A curly brace over `start`..`end`, as a `Brace(points, tip)`. Its ends lie on the line `base` (a constant across coordinate), and the tip sits at the middle of the span, `depth` past `base` on the side `direction` (± 1). On axis `"y"` points are (across, along); on `"x"` they are (along, across). Each quarter arc is sampled with `samples` segments. An empty span, a non-positive depth or another direction or axis raise `ValueError`.

The outline is built in a local frame, u across and v along, from four quarter circles of radius $r = \min\left(\frac{\text{depth}}{2}, \frac{\text{end} - \text{start}}{4}\right)$ joined by two straight pieces, and then stretched across by depth $/2r$, so a short span gets small arcs but the tip still reaches the full depth (Figure 6).

Proposition 20 (Shape of a brace). *Let $\ell < h$ be the ends of the span, $m = (\ell + h)/2$ its middle and δ the depth. In the local frame, before sampling, the outline is a curve from $(0, \ell)$ to $(0, h)$ through the tip (δ, m) that (i) stays in the strip $0 \le u \le \delta$, (ii) is symmetric under $v \mapsto \ell + h - v$, and (iii) has a continuous tangent everywhere except at the tip, where it turns back on itself (a cusp).*

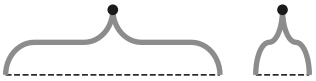


Figure 6
Brace outlines; the right span is under 2δ.

8 Region and point labels

A label identifies a filled region or point that has already been drawn. Deferred passes draw label layers after every other layer, when all obstacles are known. Point labels are placed before region labels, and each placed label becomes an obstacle for subsequent labels.

8.1 Region labels

`RegionLabelLayer` `RegionLabelLayer(region, text, short_text=None, placement="auto", style=None, stroke=Stroke(width=0.8), *, role="text", ...)`

New: 0.2.0 Labels a filled region, specified by the id of a `FillLayer` or by a polygon with at least three points. `stroke` sets the callout leader style.

`placement="auto"` Inside the region when the text fits, centred on its pole (Definition 17), trying `short_text` second; otherwise a callout with `text`.

`placement="inside"` As `"auto"`, but when neither text fits, `short_text` (or `text`) is centred on the pole regardless.

`placement="callout"` Always a callout: unboxed text outside the region and a thin leader from the pole to the text.

A text fits inside when its rectangle, padded by 2 pt on each side and centred on the pole, lies in the region (Theorem 14) and touches no line, marker, text or other region. Figure 7 shows both outcomes.

8.2 Point labels

`PointLabelLayer` `PointLabelLayer(point, text, style=None, *, role="text", ...)`

New: 0.3.0 Places text beside a point without a leader. The point's *footprint* is its marker when one is drawn there, or the bare point otherwise.

The label tries 16 directions around the footprint at gaps of 4, 7, 11 and 16 pt from its edge, starting with the nearest gap. Within each gap, it tries the directions in the order defined by Definition 21 (Figure 8). It uses the first position that covers nothing. If every position has violations, it uses the earliest position among those with the fewest and emits a `Layout`

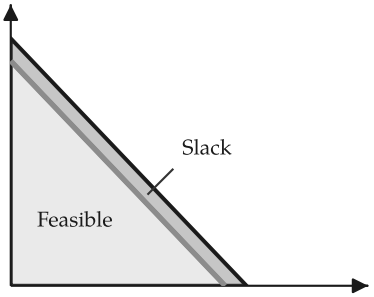


Figure 7
An inside label and a callout.

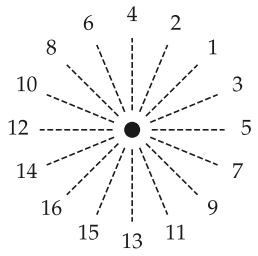


Figure 8
The order in which directions are tried.

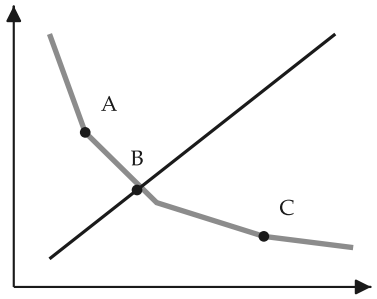


Figure 9
Point labels beside their points.

`Warning` naming the layer. A region is not an obstacle when the point lies strictly inside it, more than 1.5 px from its edge, because that is where the label belongs.

Definition 21 (Candidate order). Number the directions $k = 0, \ldots, 15$ counter-clockwise from $+x$, at angles $2\pi k/16$. Point labels try them in increasing order of the key $(\min(s, 16 - s), [s > 8])$ with $s = (k - 2) \bmod 16$: by angular distance from the upper-right direction $k = 2$, and among two directions at the same distance, the counter-clockwise one first.

8.3 Hard constraints

Both kinds of label must cover nothing. Obstacles are collected from the content on the axes: path segments, marker and text rectangles, and filled region polygons. The searches below are defined in `mosaickit.layout.placement`; all except `place_beside` are also exported from `mosaickit.layout`. Like the geometry functions, they work in display pixels. `scale` converts their point-based gaps to pixels.

`Obstacles` `Obstacles(segments=(), rects=(), polygons=())`
`Placement` `Placement(rect, leader, violations)`

`Obstacles` records what a label must avoid. `Placement` records the resulting rectangle, the leader segment (`None` for a label without one) and the number of violated constraints. `Obstacles.extended(segments=..., rects=...)` adds the label just placed.

Definition 22 (Violations). For a candidate rectangle R, obstacles O, plot bounds B and a set of polygons $\mathcal{P}$, the *violation count* is

$$V(R) = [R \not\subseteq B] + \#\{s \in O_{\text{seg}} : R \cap s \neq \emptyset\} + \#\{Q \in O_{\text{rect}} : R \cap Q \neq \emptyset\} + \#\{P \in \mathcal{P} : R \cap \overline{P} \neq \emptyset\},$$

where $[\cdot]$ is 1 for a true condition and 0 otherwise. A candidate *covers nothing* when $V(R) = 0$.

Each term is computed exactly by Lemma 10, by `Rect.intersects` and by Corollary 15.

8.4 Callouts

`place_callout` `place_callout(polygon, size, obstacles, bounds, *, scale=1.0) -> Placement`

New: 0.2.0 Searches around a region for a callout of the given size. For each of 16 directions r from the pole o, it places a candidate rectangle at distance `ray_exit(o, r, P)` plus a gap of 12, 24 or 40 pt (the near ring). The side or corner facing the pole anchors the rectangle, so it extends away from the region. The leader runs from the pole toward the rectangle's nearest point (Lemma 6) and stops 2 pt short. The charge for a candidate starts at $V(R)$, with the region itself included among the polygons. It increases by one if the centre is not in open space, by one for each line, text or other region crossed by the leader after it leaves its own region, and by one for each crossing beyond which the candidate lies (Definition 23).

Definition 23 (Open space and crossings). *Open space* is the union of the connected free areas, on a 4 pt grid of the plot, that cover at least a tenth of it; smaller free areas are *pockets*, such as an unfilled strip between two shaded regions, and text there would seem to name the pocket. A *crossing* of a region is a point x where two obstacle segments cross properly (each passes strictly through the other) on or within 1.5 px of the region's boundary. A point p lies *beyond* x, seen from the pole o, when $(p - x) \cdot (x - o) > 0$.

Past a crossing, the lines diverge away from the region. A label there would appear to describe their extensions instead of the region. Orientation tests locate the crossing point.

Lemma 24 (Crossing point). *If $[a, b]$ and $[c, d]$ cross properly, they meet at the single point $a + t(b - a)$ with*

$$t = \frac{(c - a) \times (d - c)}{(b - a) \times (d - c)} \in (0, 1),$$

where $u \times v = u_x v_y - u_y v_x$.

Proposition 25 (Choice of a callout). *Order candidates by the key (charge, leader length), compared lexicographically, and enumerate them direction by direction, nearest gap first. If some candidate in the near ring has charge 0, `place_callout` returns the first such candidate with the shortest leader. Otherwise it also tries the far ring (gaps of 60 and 90 pt) and returns the first candidate with the smallest key over both rings. The result depends only on the arguments.*

When the returned charge is not zero, the renderer still draws the callout there and emits a `LayoutWarning` naming the layer; enlarging the canvas or shortening the text usually removes it.

`fits_inside` `fits_inside(polygon, size, obstacles, bounds) -> Rect | None`

`place_point_label` `place_point_label(footprint, size, obstacles, bounds, *, scale=1.0) -> Placement`

`place_beside` `place_beside(anchor, size, *, axis, direction, reach, obstacles, bounds, scale=1.0)`

`fits_inside` returns a rectangle centred on the pole if it fits and touches nothing except its own region. `place_point_label` performs the point-label search described above. `place_beside` searches beside a brace tip: it tries gaps of 3 to 45 pt beyond the tip and slides along the span by up to `reach` in 3 pt steps, trying the smallest shift first. It returns the rectangle and its violation count. If the brace label has no free position there, the renderer tries a callout from the tip and keeps the result with fewer violations.

9 Styles and colors

9.1 Sparse styles

Every style is a frozen dataclass whose fields may all be `None`. `None` means inherit: the value comes from a style below it (Section 10). Falsy values are not `None`, so `opacity=0`, `width=0` and `LegendStyle(visible=False)` are explicit overrides.

Definition 26 (Sparse merge). Let a and b be styles of the same type with fields F. The style $a \rhd b$ (`a.merged_over(b)`) has, for each $\varphi \in F$,

$$(a \rhd b)_\varphi = \begin{cases} a_\varphi \text{ if } a_\varphi \neq \text{None} \\ b_\varphi \text{ otherwise.} \end{cases}$$

The empty style ε has every field `None`. For style bundles the merge is taken slot by slot, a `None` slot acting as ε.

Proposition 27 (Merging is a monoid). *For styles a, b, c of one type, (i) associativity states*

$$(a \rhd b) \rhd c = a \rhd (b \rhd c);$$

(ii) the empty style is an identity,

$$\varepsilon \rhd a = a \rhd \varepsilon = a;$$

and (iii) merging is idempotent,

$$a \rhd a = a.$$

Field by field, $a_1 \rhd \cdots \rhd a_n$ takes the first value that is not `None`.

A stack of styles can therefore be merged in any grouping. The result acts as a priority list: the first source that sets a field wins.

`SparseStyle` `style.merged_over(base)`

The merge of Definition 26. Styles of different types raise `TypeError`.

9.2 Style types

`Stroke` `Stroke(color=None, width=None, dash=None, arrow=None, opacity=None)`

Line styles. `width` is in points; `dash` is a `DashStyle` (`SOLID`, `DASHED`, `DOTTED`, `DASHDOT`); `arrow` is an `ArrowStyle` (`OPEN`, `TRIANGLE`, `FANCY`, `WEDGE`) drawn at the layer's `ArrowPlacement` (`START`, `END`, `BOTH`).

`DashStyle` `DashStyle.SOLID`, `DASHED`, `DOTTED`, `DASHDOT`

Line patterns of a `Stroke`. The short forms `-`, `--`, `:` and `-.` are accepted too.

`SOLID` *DashStyle* Solid line.

`DASHED` *DashStyle* Dashed line.

`DOTTED` *DashStyle* Dotted line.

`DASHDOT` *DashStyle* Dash-dot line.

`ArrowStyle` `ArrowStyle.OPEN`, `TRIANGLE`, `FANCY`, `WEDGE`

Arrowheads of a `Stroke`. `SIMPLE` is another name for `OPEN`, and the Matplotlib spellings `->` and `-|>` are accepted.

`OPEN` *ArrowStyle* Open arrowhead.

`TRIANGLE` *ArrowStyle* Filled triangle.

`FANCY` *ArrowStyle* Curved, tapering arrowhead.

`WEDGE` *ArrowStyle* Thin wedge-shaped arrowhead.

`ArrowPlacement` `ArrowPlacement.START`, `END`, `BOTH`

Where a layer's arrowhead is drawn.

`START` *ArrowPlacement* At the first point.

`END` *ArrowPlacement* At the last point.

`BOTH` *ArrowPlacement* At both ends.

`Fill` `Fill(color=None, opacity=None, hatch=None)`

Styles for region interiors. `hatch` is a Matplotlib hatch pattern such as `"//"`; `""` means no hatch.

`Marker` `Marker(color=None, size=None, shape=None, opacity=None, edge_color=None, edge_width=None)`

Point-marker styles. As in Matplotlib's `scatter`, `size` is the area in square points (36 is a 6 pt disc). `shape` is a Matplotlib marker such as `"o"`, `"s"` or `"X"`.

`TextStyle` `TextStyle(color=None, size=None, family=None, weight=None, opacity=None, rotation=None)`

Text styles. `size` is in points, `family` is a font-family name, `weight` may be a value such as `"bold"`, and `rotation` is in degrees counter-clockwise.

`LegendStyle` `LegendStyle(visible=None, location=None, frame=None, size=None)`

Legend styles, including a Matplotlib location such as `"best"` or `"upper right"`, a frame setting and the font size.

Sizes, widths and opacities are checked when a style is created: sizes must be finite and non-negative, opacities in $[0, 1]$, rotation finite.

Table 4 Primitive defaults

Style	Values
`Stroke`	`grey-800`, width 1.5, solid, opacity 1
`Fill`	`grey-200`, opacity 0.3, no hatch
`Marker`	`grey-800`, size 36, `"o"`, opacity 1, edge `grey-800` of width 0
`TextStyle`	`grey-900`, 12 pt, DejaVu Sans, normal weight, opacity 1, no rotation
`LegendStyle`	visible, location `"best"`, no frame, 10 pt

9.3 Colors

`Color` `Color(red, green, blue, alpha=1.0)`
`TRANSPARENT` `Color.from_hex("#RGB" | "#RRGGBB" | "#RRGGBBAA")`

An immutable RGBA color with channels in $[0, 1]$. `channels` returns the four values, `from_channels()` builds one, and `to_hex(include_alpha=None)` writes `#RRGGBB`, adding `AA` when `include_alpha` is true, or by default when alpha is not 1. `TRANSPARENT` is `Color(0, 0, 0, 0)`.

Proposition 28 (Hex round trip). *For every string h of the form `#RRGGBB` or `#RRGGBBAA` in hexadecimal digits, `Color.from_hex(h).to_hex(include_alpha=len(h) == 9)` equals h in upper case. A three-digit `#RGB` is read as `#RRGGBB`.*

A style `color` or `edge_color` takes a `Color`, a `"#hex"` string (parsed at once) or any other string, which is kept as a *palette name*.

9.4 Palettes

`Palette` `Palette(name, colors)`
`DEFAULT_PALETTE`

New: 0.2.0 A named table of colors. Values may be `Color`s or hex strings. `palette[name]` looks up a color and raises `ConfigurationError`, naming the palette, when the name is unknown. `name in palette` tests whether a name exists.

Table 5 `DEFAULT_PALETTE`

Name	Value	Used by the default theme for
`grey-900`	`#222222`	Text
`grey-800`	`#333333`	Lines, markers, axes
`grey-600`	`#666666`	Axis notes
`grey-400`	`#999999`	Guides
`grey-200`	`#CCCCCC`	Fills
`grey-100`	`#E6E6E6`	
`white`	`#FFFFFF`	The canvas background
`blue`	`#01A2D9`	`primary`
`red`	`#E3120B`	`secondary`
`teal`	`#00887D`	`accent`

Themes and styles name their colors, and the names are looked up in the active palette (`Config.palette`) when a canvas builds its render plan, so renderers only ever see concrete colors. Changing a palette entry recolors every role that names it. A missing name raises `ConfigurationError` at render time, naming the role, the style field and the palette. A Python `Palette` replaces the default palette, so build it from `DEFAULT_PALETTE.colors` to keep the names the built-in theme uses:

```
from mosaickit import DEFAULT_PALETTE, Canvas, Config, Palette, use_config

brand = Palette("brand", {**DEFAULT_PALETTE.colors, "blue": "#0072B2"})
with use_config(Config(palette=brand)):
    canvas = Canvas()          # primary now draws in #0072B2
```

10 Themes and configuration

10.1 Style bundles and themes

`StyleBundle` `StyleBundle(stroke=None, fill=None, marker=None, text=None, legend=None)`

One sparse style per slot. A slot holding the wrong type raises `ConfigurationError`. `merged_over(base)` merges slot by slot (Definition 26).

`Theme` `Theme(name, roles)`

A theme name and an immutable mapping from role names to bundles. Role names are non-empty dotted strings (`"axes"`, `"axes.note"`, `"mypkg.boundary"`). `with_roles(**patch)` returns a theme with each patched role merged over its old bundle; since keyword names cannot contain dots, pass dotted roles as `with_roles(**{"mypkg.line": bundle})`. `resolve(role, fallback_category=...)` resolves a role against the theme alone.

Table 6
The `default` theme

Role	Bundle
`primary`	stroke `blue`
`secondary`	stroke `red`
`accent`	stroke `teal`
`guide`	stroke `grey-400`, dashed
`axes`	stroke `grey-800`, width 1
`axes.note`	text 9 pt, `grey-600`
`canvas`	fill `white`, opacity 1 (the background)

Every other value comes from the primitive defaults of Table 4. The theme is `mosaickit.themes.default`.

10.2 Role resolution

A layer's role is resolved through its ancestor roles and then through the fallback category for its layer type. For each field, the fallback applies only when neither the role nor any ancestor supplies a value.

Definition 29 (Role chain). The *chain* of a role $\rho = \rho_1.\rho_2\cdots\rho_m$ with fallback category φ is the sequence

$$\kappa(\rho) = (\rho_1\cdots\rho_m, \rho_1\cdots\rho_{m-1}, \ldots, \rho_1, \varphi),$$

most specific first, with a repeated key kept only at its first place.

For example `"axes.note"` on a text layer has the chain (`axes.note`, `axes`, `text`). The fallback categories are `primary` (paths, groups), `region` (fills), `point` (markers), `text` (text, labels, marks, notes, braces), `annotation` (arrows) and `legend`.

Theorem 30 (Resolution order). *Let a layer with explicit styles E and role chain $\kappa = (k_1, \ldots, k_n)$ be drawn on a canvas with role overrides C, under a configuration with role overrides G and theme roles T, and let D be the primitive defaults. Then each field of the layer's resolved style is the first value that is not* `None` *in the sequence*

$$E, \quad C[k_1], \ldots, C[k_n], \quad G[k_1], \ldots, G[k_n], \quad T[k_1], \ldots, T[k_n], \quad D,$$

where a missing role counts as the empty bundle.

Corollary 31 (Overrides beat specificity). *A canvas or configuration override of a general role takes precedence over the theme's value for a more specific role: if G*[text] *sets a text size and no source before it in Theorem 30 does, a text layer with role* `axes.note` *gets that size, not the theme's 9 pt.*

The figures in this manual use Corollary 31. They override the text size of `text`, `axes` and `axes.note` together because overriding `text` alone would also enlarge the notes.

10.3 Registering domain roles

`ThemeRegistry` `ThemeRegistry()`

A registry of themes, holding `default` from the start. `register(theme)` adds one (a taken name raises `ConfigurationError`), `get(name)` looks one up, and `register_roles(name, roles)` merges dotted domain roles into a registered theme and returns it; an undotted role raises `ConfigurationError`, so domain packages cannot shadow the built-in roles.

`RolePack` `expand_roles(pack) -> dict[str, StyleBundle]`
`expand_roles`

A typed representation of a domain package's roles. A `RolePack` is a dataclass with a class variable `_namespace`; each field that is not `None` becomes the role `namespace.field`, with underscores in the field name turned into dots.

```
from dataclasses import dataclass
from typing import ClassVar
from mosaickit import Stroke, StyleBundle, Theme, expand_roles
from mosaickit.themes import default

@dataclass(frozen=True)
class MyRoles:
    _namespace: ClassVar[str] = "mypkg"
    line: StyleBundle | None = StyleBundle(stroke=Stroke(color="blue",
width=2))
    line_dashed: StyleBundle | None = None

roles = expand_roles(MyRoles())        # {"mypkg.line": ...}
mytheme = Theme("mypkg", {**default.roles, **roles})
```

10.4 Configuration

`Config` `Config(theme=default, canvas_spec=CanvasSpec(), renderer="matplotlib", role_overrides={}, palette=DEFAULT_PALETTE)`

Runtime defaults for new canvases: the theme, the specification, the renderer (a name or a `Renderer`), role overrides applied after the theme (G in Theorem 30) and the palette.

`use_config` `with use_config(config): ...`

Makes `config` the active configuration inside the block. A context variable stores the value, so each thread and asynchronous task sees its own configuration. Arguments passed to `Canvas` always take precedence.

`Config.load` `Config.load(path)`
`Config.from_dict` `Config.from_dict(data, *, source="<dict>")`

Reads a strict TOML configuration. Top-level keys are `theme` (only `"default"`; custom themes are passed in Python), `canvas` (fields of `CanvasSpec`), `renderer` (only `"matplotlib"`), `palette` and `styles`. Any other key, unknown style, bad value or unreadable file raises `ConfigurationError` naming the file and the key.

```toml
[canvas]
x_range = [0, 20]
dpi = 150

[palette]
blue = "#0072B2"      # override a default color
accent = "#984EA3"    # add a new name

[styles."mypkg.boundary".stroke]
color = "accent"      # a palette name or a "#hex" value
width = 2
dash = "dashed"
```

The `[palette]` table is merged over `DEFAULT_PALETTE`, so overriding `blue` there recolors `primary` and everything else that names `blue`. Style `color` and `edge_color` values may be palette names; an unknown name raises `ConfigurationError` naming the file and key when the file is loaded, before any canvas renders.

11 Parameters, grids and animation

11.1 Parameters and expressions

A layer coordinate may be an expression in named parameters instead of a number. The resulting scene describes a family of diagrams, one for each set of parameter values. Binding selects one diagram from that family.

`Parameter` `Parameter(name, value_type=None)`

A named placeholder. With `value_type` set, a bound value must be an instance of it; every bound value must be hashable. `values(seq)` returns `ParameterValues`, checking each value at once.

`Expression` `Constant(value)`
`Constant`

Immutable expression trees. Parameters and constants combine with `+`, `-`, `*`, `/`, `**`, unary `-` and the comparisons `<`, `<=`, `>`, `>=`, and `equals()`; plain numbers are wrapped as constants. Expression equality is structural: `==` compares trees, while `equals()` builds a comparison expression. Using an expression as a truth value raises `BindingError`; evaluate it first. `evaluate(bindings)` computes the value from a mapping of parameters to values, and `free_parameters()` returns the set of parameters in the tree.

Definition 32 (Expressions and binding). An *expression* is a constant, a parameter, or $e_1 \circ e_2$ for expressions e_1, e_2 and an operation $\circ$. Its free parameters satisfy

$$\operatorname{free}(c) = \emptyset, \quad \operatorname{free}(p) = \{p\},$$

and

$$\operatorname{free}(e_1 \circ e_2) = \operatorname{free}(e_1) \cup \operatorname{free}(e_2).$$

A *binding* β is a finite map from parameters to values. $\operatorname{bind}(e, \beta)$ is the value $e(\beta)$ if $\operatorname{free}(e) \subseteq \operatorname{dom} \beta$; it is e itself if e is an unbound parameter; otherwise it is

$$\operatorname{bind}(e_1, \beta) \circ \operatorname{bind}(e_2, \beta).$$

A plain value v has no free parameters and evaluates to itself.

Theorem 33 (Partial binding). *For every expression e and binding β, (i)*

$$\operatorname{free}(\operatorname{bind}(e, \beta)) = \operatorname{free}(e) \setminus \operatorname{dom} \beta;$$

and (ii), for every binding γ satisfying

$$\operatorname{dom} \gamma \cap \operatorname{dom} \beta = \emptyset$$

and

$$\operatorname{dom} \gamma \supseteq \operatorname{free}(e) \setminus \operatorname{dom} \beta,$$

evaluation obeys $\operatorname{bind}(e, \beta)(\gamma) = e(\beta \cup \gamma)$.

Corollary 34 (Binding in stages). *For bindings β_1, β_2 with disjoint domains,* $\operatorname{bind}(\operatorname{bind}(e, \beta_1), \beta_2)$ *and* $\operatorname{bind}(e, \beta_1 \cup \beta_2)$ *have the same free parameters and the same value under every binding of them. In particular* `canvas.bind(p, 1).bind(q, 2)` *draws the same diagram as* `canvas.bind({p: 1, q: 2})`.

`bind` walks through tuples, mappings and every `mosaickit` dataclass, so a whole scene is bound at once; a layer's `model` is left alone. Rendering a scene that still has free parameters raises `BindingError` naming them.

```
from mosaickit import Canvas, Parameter, TextLayer

x = Parameter("x", value_type=float)
template = Canvas().add(TextLayer((x, 2 * x + 1), "moving"))
frame = template.bind(x, 3.0)
print(frame.snapshot().layers[0].position)    # (3.0, 7.0)
```

11.2 Grids

`CanvasGrid` `CanvasGrid(cells, rows=None, cols=None, shape=None, links=())`

`Span` `Span(canvas, rows=1, cols=1)`

Places several canvases in one figure. `cells` is either a flat list of canvases, `Spans` and `Nones` (empty cells), filled row by row, or a list of rows, where row spans are allowed and every row must account for every column. `shape=(rows, cols)` is the same as passing both. Each canvas is copied, so changing it afterwards does not change the grid. Overlapping spans, spans past the edge, too many cells or mixed flat and nested cells raise `ConfigurationError`. `render()` and `save()` work as on a canvas.

Proposition 35 (Inferred grid shape). *For a flat list of $n \geq 1$ plain cells with neither* `rows` *nor* `cols` *given, the grid has $c = \lceil \sqrt{n} \rceil$ columns and $r = \lceil n/c \rceil$ rows. Then $rc \geq n$, $r \leq c$, and fewer than c cells are left empty, all in the last row.*

With only `cols` given, $r = \lceil n/c \rceil$; with only `rows`, $c = \lceil n/r \rceil$.

Layout
CanvasGrid.from_layout

```
CanvasGrid.from_layout(canvases, layout)
```

Common arrangements: `SINGLE`, `STACKED` (two rows), `SIDE_BY_SIDE`, `GRID_2X2`, `GRID_3X3`, and the three-canvas `TOP_TWO_BOTTOM_ONE` and `TOP_ONE_BOTTOM_TWO`, whose single canvas spans both columns.

CanvasGrid.sweep

```
CanvasGrid.sweep(template, values, *, cols=None)
```

Creates one cell for each value in the supplied `ParameterValues`, with the template bound to that value.

GridLink

```
GridLink(start_cell, start, end_cell, end, role="link", stroke=None)
```

New: 0.5.0

A straight line from `start` in one cell to `end` in another, each in its own cell's data coordinates, drawn over the figure across the gaps between cells. Cells are numbered in placement order. Its style is `role` resolved in the start cell's theme, with `stroke` on top. A cell index out of range raises `ConfigurationError` when the grid is built.

```
from mosaickit import Canvas, CanvasGrid, GridLink, MarkerLayer,
Parameter, Stroke

shift = Parameter("shift", value_type=float)
template = Canvas().add(MarkerLayer([(5, 2.5 + shift)]))
cells = [template.bind(shift, v) for v in (0.0, 2.0, 4.0)]
link = GridLink(0, (5, 2.5), 2, (5, 6.5), stroke=Stroke(dash="dashed"))
CanvasGrid(cells, cols=3, links=[link]).save("sweep.pdf")
```

11.3 Animation

Animation

```
Animation(template, parameter, values, fps=30)
Animation.sweep(canvas, values, *, fps=30)
```

Creates one frame per value by binding the parameter in the template. `frames()` yields the frames as renderer-neutral scenes; `save(path)` writes a GIF (through Pillow) or an MP4 (through `ffmpeg`). The values are checked against the parameter when the animation is created; an empty list or a non-positive `fps` raises `ConfigurationError`.

```
from mosaickit import Animation

values = shift.values([0.0, 1.0, 2.0, 3.0])
Animation.sweep(template, values, fps=4).save("sweep.gif")
```

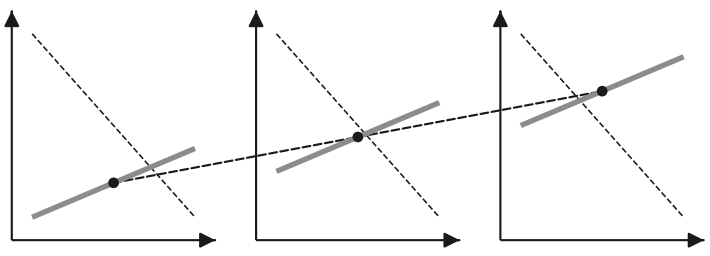


Figure 10
A three-value sweep with a link from the first cell to the last.

12 Rendering

12.1 Renderers

`Renderer`

```
name: str
render(scene, context) -> Any
save(result, target, options) -> list[Path]
```

The protocol implemented by every backend. A renderer receives an immutable scene and a private render context (specification, theme, overrides, palette, cache and bindings), and returns a result that `save` writes. Grids and animations also require `render_grid` and `save_animation`; a renderer without them raises `RenderError`. Render plans and contexts remain private while the built-in backend evolves.

`RendererRegistry` `RendererRegistry()`

`register(renderer)` adds a backend under its `name`. An object missing `name`, `render` or `save`, or one whose name is already registered, raises `RenderError`. `get(name)` returns a registered renderer or loads a built-in one. The only built-in is `"matplotlib"`, imported on first use (Hunter, 2007).

12.2 The Matplotlib renderer

Rendering a canvas builds a render plan: it binds the scene (a free parameter raises `BindingError`), flattens groups, sorts layers by `z_index`, resolves each layer's style (Theorem 30) and replaces palette names by colors. The figure gets the canvas size and DPI, the `canvas` role's fill as its background, and axes set to the specification's ranges with Matplotlib's own axis lines turned off; axes in mosaickit are layers.

Builders, one per layer type, then draw the layers in order. Deferred passes handle layer types whose work depends on everything else: legends, point labels, region labels, gutter text and span braces. These passes run in registration order, and each receives all layers of its type at once.

`register_builder` `register_builder(layer_type, builder)`
`register_pass` `register_pass(layer_type, run)`

New: 0.2.0 These functions in `mosaickit.rendering.matplotlib` teach the renderer a new layer type without changing mosaickit. A builder is called as `builder(ax, resolved)` and returns the Matplotlib artist; a pass is called as `run(ax, layers, context)` with a `PassContext` whose `handles` map layer ids to legend artists. `resolved.layer` is the layer and `resolved.style` its resolved `StyleBundle`. Lookups follow the method resolution order, so a subclass inherits its parent's registration. A layer declares which of its fields holds its explicit style for each slot through the class variable `style_slots`, and its last-resort role through `fallback_category`.

Region-label callouts avoid everything on the axes, including what registered third-party builders drew.

12.3 Results and saving

`SaveOptions` `SaveOptions(transparent=False, expand=True)`

Controls how a result is written; `canvas.save(path, **options)` passes its keyword arguments here. `transparent` drops the background. With `expand=True`, saving grows the canvas just enough, plus a 4 pt margin, to include anything drawn past its edges, such as gutter text; it never crops, so a diagram that fits keeps exactly the size of its `CanvasSpec`. `expand=False` keeps the specified size.

```
result = canvas.render()
try:
    result.show()          # an interactive window
finally:
    result.close()
```

`render()` returns a `MatplotlibResult` with `figure`, `axes`, `show()`, `save(target, **options)` and `close()`. `canvas.save()` closes its temporary result itself. The file format follows the extension: `.png`, `.pdf` or `.svg` (anything else raises `RenderError`); animations take `.gif` or `.mp4`. A grid gives every cell one size: the largest canvas width divided by the columns it spans, by the largest canvas height divided by the rows it spans; it uses the highest DPI among its canvases.

12.4 Caching

`RenderCache` `RenderCache(max_entries=256)`
`CacheKey`

A bounded, thread-safe least-recently-used cache for resolved layers, keyed by layer id, bindings, model, specification and style. Each render, grid or animation creates its own cache unless one is passed with `cache=`, so jobs never share state by accident; pass the same cache to reuse work across jobs. A model that cannot be hashed is rendered without caching, with one `CacheBypassWarning` per model type and cache; use frozen dataclasses for models. `clear()` empties the cache.

Appendix A Proofs

This appendix proves the lemmas, propositions, theorems and corollaries in the order they appear in the chapters. The proofs assume exact arithmetic. The code computes the same quantities in floating point, so it may decide a configuration either way when it is within rounding error of a degenerate case, such as a point on an edge or three nearly collinear points. For plane vectors u, v, define $u \times v = u_x v_y - u_y v_x$.

A.1 Packing and lanes

Proof of Theorem 2. Change of variables. Put $o_k = \sum_{j<k} \big(s_j + g\big) + s_k/2$, $y_k = x_k - o_k$ and $z_k = c_k - o_k$. Since $o_{k+1} - o_k = (s_k + s_{k+1})/2 + g$, the constraints of Definition 1 read $y_1 \le \cdots \le y_n$, and the objective is $\sum_k (y_k - z_k)^2$. The problem is therefore to find the closest non-decreasing vector to z in the least-squares sense.

What the code computes. A cluster B of consecutive items starting at i, with start S_B, places item $k \in B$ at

$$x_k = S_B + \sum_{i \le l < k} (s_l + g) + s_k/2 = S_B + o_k - O_i,$$

where $O_i = \sum_{l<i}(s_l + g)$. Hence $y_k = S_B - O_i =: Y_B$ is constant on B, and the code's choice of S_B as the mean of $c_k - (o_k - O_i)$ makes Y_B the mean of z_k over B; a new item alone has $Y = z_k$. Clusters A and B (with B after A) are left apart when $S_A + \text{length}(A) + g \le S_B$; since $\text{length}(A) + g = O_{i_B} - O_{i_A}$, this says $Y_A \le Y_B$. The code is therefore the pool-adjacent-violators algorithm: append z_k as a block, and while the last two blocks have decreasing means, replace them by their union with its mean.

Invariants. (a) Consecutive blocks have non-decreasing means: merging stops exactly when the last pair is in order, and earlier pairs are not touched. (b) In every block B with mean μ_B, every initial segment has mean at least μ_B. A single item satisfies (b). When A and B merge, $\mu_A > \mu_B$, and the merged mean μ lies strictly between them. An initial segment inside A has mean at least $\mu_A > \mu$. Any other initial segment is A followed by an initial

segment B' of B; if B' is all of B its mean is μ, and otherwise the rest B'' of B has mean at most $\mu_B < \mu$ by (b) for B, so the segment, being the complement of B'' in a set with mean μ, has mean at least μ.
Optimality. The objective is strictly convex and the constraints $y_k - y_{k+1} \le 0$ are linear, so a point satisfying the Karush–Kuhn–Tucker conditions is the unique minimizer (Boyd & Vandenberghe, 2004). Let $\lambda_k = 2\sum_{l \le k}(z_l - y_l)$ for $k = 0, \dots, n$. Stationarity, $2(y_k - z_k) + \lambda_k - \lambda_{k-1} = 0$, holds by construction. Within a block the deviations $z_l - Y_B$ sum to zero, so λ is zero at every block end, $\lambda_n = 0$ included. Inside a block starting at i, $\lambda_k = 2\sum_{l=i}^{k}(z_l - Y_B) \ge 0$ by (b). Finally $\lambda_k > 0$ only when k and $k+1$ lie in the same block, where $y_k = y_{k+1}$ and the constraint is tight. With (a) for feasibility, all conditions hold. □

Proof of Corollary 3. (i) Number the items in sorted order and let $i < j$. Summing the constraints from i to $j-1$,

$$x_j - x_i \ge \sum_{k=i}^{j-1}((s_k + s_{k+1})/2 + g) \ge \left(s_i + s_j\right)/2 + g$$

since every term is non-negative and the first and last contribute $s_i/2$ and $s_j/2$. (ii) If c is a packing, it attains the objective 0, so it is the unique minimizer of Theorem 2. (iii) With the notation of that proof,

$$\sum_{k \in B}(x_k - c_k) = \sum_{k \in B}(y_k - z_k) = |B|\, Y_B - \sum_{k \in B} z_k = 0.$$

□

Proof of Theorem 5. An interval joins a lane only when it conflicts with nothing in it, and it never moves afterwards, so the result is a lane assignment. Any lane assignment puts ω pairwise conflicting intervals in ω distinct lanes, so at least ω lanes are needed.
Conversely, write $J = \left[a_J, b_J\right]$ and suppose the intervals come in increasing order of a. By Definition 4, I and J conflict exactly when the half-open intervals $[a_I, b_I + g)$ and $\left[a_J, b_J + g\right)$ meet. Let I be an interval placed in the highest lane k used. Each lane $j < k$ held, when I came, an interval J_j conflicting with I. Each J_j came earlier, so $a_{J_j} \le a_I$, and conflicting with I it has $b_{J_j} + g > a_I$. Hence every $\left[a_{J_j}, b_{J_j} + g\right)$ contains a_I, and so does $[a_I, b_I + g)$ because $b_I - a_I + g > 0$. Half-open intervals sharing a point meet pairwise, so $J_0, \dots, J_{k-1}, I$ are $k+1$ pairwise conflicting intervals and $k + 1 \le \omega$. □

A.2 Rectangles, segments and polygons

Proof of Lemma 6. $|q - p|^2 = (q_x - p_x)^2 + \left(q_y - p_y\right)^2$, and R is the product of $[x_0, x_1]$ and $[y_0, y_1]$, so the two coordinates are minimized separately. On an interval, $t \mapsto (t - p_x)^2$ is strictly convex with unconstrained minimum at p_x, so its minimum over $[x_0, x_1]$ is attained only at $\min(\max(p_x, x_0), x_1)$; likewise for y. □

Proof of Lemma 8. Subtracting the first row from the others,

$$\operatorname{orient}(a, b, c) = \det\begin{pmatrix} 1 & a_x & a_y \\ 1 & b_x & b_y \\ 1 & c_x & c_y \end{pmatrix}.$$

A cyclic permutation of the rows is even and a swap is odd, which gives the symmetries. Also

$$\operatorname{orient}(a, b, c) = (b - a) \times (c - a) = |b - a|\,|c - a| \sin\theta,$$

where θ is the signed angle from $b - a$ to $c - a$; it is positive exactly when $\theta \in (0, \pi)$, that is when c is left of the directed line, negative when it is right, and zero exactly when the two vectors are parallel or one vanishes, that is when the points are collinear. □

Proof of Theorem 9. First observe that if $\text{orient}(c, d, p) = 0$ and p lies in the bounding box of $[c, d]$, then $p \in [c, d]$. Indeed, if $c = d$ the box is the point c; otherwise $p = c + t(d - c)$ for some real t by collinearity, and on a coordinate where $d - c$ is non-zero the box condition forces $t \in [0, 1]$.
If. Suppose $d_1 d_2 < 0$ and $d_3 d_4 < 0$. The orientation $\text{orient}(c, d, \cdot)$ is affine along $[a, b]$ and changes sign, so $[a, b]$ meets the line L_{cd} at one point; likewise $[c, d]$ meets L_{ab}. The lines are not parallel, since a and b lie on opposite sides of L_{cd}, so they share exactly one point z, and both crossing points equal z, which therefore lies on both segments. In the other case some $d_i = 0$ and its endpoint lies in the other segment's box, hence on the other segment by the note.
Only if. Let z be a common point. If $d_1 d_2 > 0$, then a and b lie strictly on one side of L_{cd}, and so does all of $[a, b]$, contradicting $z \in L_{cd}$; so $d_1 d_2 \leq 0$, and likewise $d_3 d_4 \leq 0$. If both are negative we are done. Otherwise some d_i is zero; say $d_1 = 0$, the other cases being symmetric. If $d_2 \neq 0$, $[a, b]$ meets L_{cd} only at a, so $z = a$ and $a \in [c, d]$: the test for d_1 succeeds. If $d_2 = 0$ too, all four points are collinear, the segments are overlapping intervals of one line, and an endpoint of one lies in the other, so one of the four tests succeeds. □

Proof of Lemma 10. The edges of R lie in R, so either test succeeding gives a common point (Theorem 9). Conversely, let $[a, b]$ meet R while neither endpoint lies in R. The segment is connected and contains points in R and outside it, so it meets the boundary of R, which is the union of the four edges; Theorem 9 detects that edge. □

Proof of Lemma 12. $t \mapsto |a + t(b - a) - p|^2$ is a convex quadratic with unconstrained minimum at $(p - a) \cdot (b - a) / |b - a|^2$; over $[0, 1]$ its minimum is at the clamp of that value, as in the proof of Lemma 6. □

Proof of Proposition 13. Write $p = (x, y)$ and choose $\varepsilon > 0$ smaller than every positive difference $|y_i - y|$ between a vertex height and y, and small enough that $p_\varepsilon = (x, y + \varepsilon)$ lies in the same component of $\mathbb{R}^2 \setminus \partial P$ as p (possible because $p \notin \partial P$ and the components are open). No vertex lies on the line $Y = y + \varepsilon$, and for every vertex $y_i > y$ if and only if $y_i > y + \varepsilon$. So an edge passes the code's height test exactly when it crosses the line $Y = y + \varepsilon$, at the abscissa

$$X_\varepsilon = x_0 + (y + \varepsilon - y_0)(x_1 - x_0)/(y_1 - y_0).$$

The code compares x with X_0. If $x = X_0$, the point (X_0, y) lies on the edge, because the height test puts y between the endpoint heights, and $p \in \partial P$, which is excluded; so $x \neq X_0$, and by continuity $x < X_0$ if and only if $x < X_\varepsilon$ for small ε. Hence the code counts the edges crossed by the rightward ray from p_ε, which passes through no vertex. Each such crossing moves the ray between the interior and the exterior, and far to the right it is outside, so the count is odd exactly when $p_\varepsilon \in \text{int } P$, that is when $p \in \text{int } P$ (Haines, 1994). □

Proof of Theorem 14. If $R \subset \text{int } P$, then R misses ∂P, so no edge touches it (Lemma 10), and its corners lie in int P, off the boundary, so they pass the even-odd test (Proposition 13). Conversely, if no edge touches R, then $R \cap \partial P = \emptyset$ (Lemma 10); in particular the corners are off the boundary, and passing the test puts them in int P. Since R is connected and misses ∂P, it lies in one component of $\mathbb{R}^2 \setminus \partial P$; it contains a corner of int P, so $R \subset \text{int } P$.□

Proof of Corollary 15. If a corner passes the test, it lies in int P when it is off the boundary (Proposition 13) and in ∂P otherwise; either way $R \cap \overline{P} \neq \emptyset$. If an edge touches R, then R meets ∂P. Conversely, suppose R meets $\overline{P}$. If it meets ∂P, some edge touches it. Otherwise R misses ∂P and meets int P, so, being connected, $R \subset \text{int } P$ and its corners pass the test.□

Proof of Proposition 16. Suppose $p = o + tr \in \partial P$ for some $t > T$, on an edge e. If e is not parallel to r, the code solves $o + tr = a + u(b - a)$ for e and finds this $t \geq 0$ with $u \in [0, 1]$,

so $T \geq t$, a contradiction. If e is parallel to r, it lies on the ray's line ℓ. Take the maximal run of consecutive edges on ℓ containing e. It is not all of P, whose vertices are not all collinear, so each end of the run is a vertex shared with an edge not on ℓ, hence not parallel to r. Consecutive edges of the run cannot double back, since two edges leaving a vertex in the same direction along ℓ would overlap and P is simple; so the run is a segment of ℓ whose far end, in the direction r, is an end vertex $v = o + t_v r$ with $t_v \geq t > T$. The adjacent non-parallel edge contains v (with $u \in \{0, 1\}$), so $T \geq t_v$, again a contradiction. Thus $\{o + tr : t > T\}$ misses ∂P; it is connected and unbounded, so it lies in the exterior. □

A.3 The pole of inaccessibility

Proof of Lemma 18. For any set S, $|d(p, S) - d(q, S)| \leq |p - q|$ by the triangle inequality, which settles the case of p, q on the same side. Let $p \in \overline{P}$ and $q \notin \overline{P}$, and let z be the last point of $\overline{P}$ on the segment from p to q (it exists since $\overline{P}$ is closed). Points just past z are outside, so $z \notin \operatorname{int} P$ and $z \in \partial P$. Then

$$|f(p) - f(q)| = d(p, \partial P) + d(q, \partial P) \leq |p - z| + |z - q| = |p - q|.$$

□

Proof of Theorem 19. The code computes f exactly: off the boundary the sign is right by Proposition 13, and on it the distance is 0. Let b denote the best value so far; it only increases, and is always f of an evaluated centre.

Termination. Each split halves the half-size h. When a cell with $h\sqrt{2} \leq \varepsilon$ is popped, $b \geq f(c)$ after the update, so its bound exceeds b by at most $h\sqrt{2} \leq \varepsilon$ and it is dropped, not split. The cells therefore form a finite tree, each pushed and popped once.

Invariant. Every point of the bounding box lies in the closed square of a cell that is either in the queue or was dropped with $f(c) + h\sqrt{2} \leq b + \varepsilon$ for the b of that moment. The initial squares, laid from the lower left corner in steps of the shorter side until they pass the far edges, cover the box; dropping keeps the invariant by the drop rule; splitting replaces a square by four that cover it.

Conclusion. At the end the queue is empty. Let p^* be a pole; it lies in $\overline{P}$, inside the box, in the square of a dropped cell with centre c and half-size h. By Lemma 18 and $|p^* - c| \leq h\sqrt{2}$,

$$f^* = f(p^*) \leq f(c) + h\sqrt{2} \leq b + \varepsilon \leq f(q) + \varepsilon,$$

where q is the returned point and $f(q)$ the final b. If $f^* > \varepsilon$ then $f(q) > 0$, so q is at positive distance from ∂P inside $\overline{P}$, that is in $\operatorname{int} P$. □

Proof of Proposition 20. Write r for the radius and $\sigma = \delta/2r \geq 1$ for the stretch. Before stretching, the curve consists of: the arc A_1 of the circle of centre $(0, \ell + r)$ from angle $-90°$ to $0°$; the segment $u = r$ from $v = \ell + r$ to $v = m - r$; the arc A_2 of centre $(2r, m - r)$ from $180°$ to $90°$; the arc A_3 of centre $(2r, m + r)$ from $270°$ to $180°$; the segment $u = r$ from $m + r$ to $h - r$; and the arc A_4 of centre $(0, h - r)$ from $0°$ to $90°$. (When $r = (h - \ell)/4$ the segments have length zero.) It starts at $(0, \ell)$, ends at $(0, h)$, and A_2 and A_3 meet at $(2r, m)$, which the stretch $(u, v) \mapsto (\sigma u, v)$ sends to (δ, m).

(i) On A_1 and A_4, $u \in [0, r]$; on the segments $u = r$; on A_2 and A_3, $u \in [r, 2r]$. After stretching, $u \in [0, \delta]$.

(ii) The reflection $v \mapsto \ell + h - v$ swaps the centres of A_1 and A_4 and of A_2 and A_3, maps angles $\theta \mapsto -\theta$, and so maps A_1 onto A_4, A_2 onto A_3, and the two segments onto each other.

(iii) Along an arc traversed with increasing angle the direction of motion is $(-\sin\theta, \cos\theta)$, and with decreasing angle $(\sin\theta, -\cos\theta)$. At the end of A_1 ($\theta = 0$) and the start of A_2 ($\theta = 180°$) both give $(0, 1)$, the direction of the segment between them; likewise at the end of A_3 and the start of A_4. At the tip, A_2 arrives with direction $(1, 0)$ ($\theta = 90°$) and A_3 leaves with $(-1, 0)$ ($\theta = 270°$): a cusp. The stretch and the final placement are invertible affine maps,

which carry tangent directions along and keep them non-zero, so continuity and the cusp survive. □

A.4 Labels

Proof of Lemma 24. A common point satisfies $a + tr = c + us$ with $r = b - a$ and $s = d - c$. Taking the cross product with s removes u: $t(r \times s) = (c - a) \times s$. A proper crossing puts a and b strictly on opposite sides of L_{cd}, so the lines are not parallel, $r \times s \neq 0$, and the point is unique. Since $\mathrm{orient}(c, d, \cdot)$ is affine along $[a, b]$ with opposite signs at the ends, its zero is at some $t \in (0, 1)$. □

Proof of Proposition 25. The code keeps one best candidate and replaces it only by a candidate with a strictly smaller key, so after any prefix of the enumeration it holds the first candidate with the smallest key seen. After the near ring, if the smallest charge is 0, the smallest key is $(0, \ell_{\min})$ and the search stops: the result is the first zero-charge candidate with the shortest leader. Otherwise the far ring is enumerated with the same best candidate, and the result is the first candidate with the smallest key over both rings. Every step is a function of the arguments (the pole, the exit distances, the open area, the crossings), with no randomness and a fixed enumeration order. □

A.5 Styles, themes and binding

Proof of Proposition 27. Per field the merge is the operation $x \circ y = x$ if $x \neq \mathrm{None}$ and y otherwise. Both $(x \circ y) \circ z$ and $x \circ (y \circ z)$ equal the first of x, y, z that is not `None` (or `None`), `None` is a two-sided identity, and $x \circ x = x$. Styles are equal when their fields are, which gives (i)–(iii), and induction gives the first-non-`None` description. For bundles the same holds slot by slot, a `None` slot behaving as the empty style. □

Proof of Proposition 28. A pair of hex digits $k \in \{0, \ldots, 255\}$ is read as $k/255$ and written as the two upper-case digits of $\mathrm{round}(255 \cdot k/255)$. In double precision the computed product differs from k by at most $255 \cdot 2^{-52} < 1/2$, so it rounds to k. The alpha pair is written exactly when `include_alpha` is true, that is for the nine-character form. A three-digit form doubles each digit before reading. □

Proof of Theorem 30. Write $\triangleright$ for the merge. Resolution starts from D and, for the mappings T, G, C in that order, merges $M[k_n], \ldots, M[k_1]$ over the result in that order (least specific first), a missing key giving the empty bundle; the layer's explicit bundle E, built from the fields named in `style_slots`, is merged last. Unwinding, and using associativity (Proposition 27) to drop the brackets, the resolved bundle is

$$E \triangleright C[k_1] \triangleright \cdots \triangleright C[k_n] \triangleright G[k_1] \triangleright \cdots \triangleright T[k_n] \triangleright D,$$

whose fields are the first non-`None` values in that order. Binding the palette afterwards replaces color names by colors and leaves the other fields alone. □

Proof of Corollary 31. A text layer with role `axes.note` has the chain (`axes.note`, `axes`, `text`), and in the sequence of Theorem 30 the entry G[text] comes before T[axes.note]. By hypothesis no earlier entry sets the size, so the first non-`None` size is that of G[text]. □

Proof of Theorem 33. By induction on e. If $\mathrm{free}(e) \subseteq \mathrm{dom}\ \beta$ (in particular for constants and for parameters in $\mathrm{dom}\ \beta$), $\mathrm{bind}(e, \beta)$ is the plain value $e(\beta)$: it has no free parameters, as (i) requires, and evaluates to $e(\beta) = e(\beta \cup \gamma)$, since the value of an expression depends only on the values of its free parameters. If e is a parameter $p \notin \mathrm{dom}\ \beta$, the result is p, with $\mathrm{free} = \{p\} = \mathrm{free}(e) \setminus \mathrm{dom}\ \beta$ and value $\gamma(p) = (\beta \cup \gamma)(p)$. Otherwise $e = e_1 \circ e_2$ and the result is $\mathrm{bind}(e_1, \beta) \circ \mathrm{bind}(e_2, \beta)$ (plain values wrapped as constants). Its free parameters

are $(\text{free}(e_1) \setminus \text{dom}\,\beta) \cup (\text{free}(e_2) \setminus \text{dom}\,\beta) = \text{free}(e) \setminus \text{dom}\,\beta$, and γ binds those of each part, so by induction its value is $e_1(\beta \cup \gamma) \circ e_2(\beta \cup \gamma) = e(\beta \cup \gamma)$. □

Proof of Corollary 34. By Theorem 33 (i) twice, both have the free parameters $\text{free}(e) \setminus \text{dom}\,(\beta_1 \cup \beta_2)$. For a binding γ of them, disjoint from β_1 and β_2, (ii) twice gives $\text{bind}(\text{bind}(e, \beta_1), \beta_2)(\gamma) = \text{bind}(e, \beta_1)(\beta_2 \cup \gamma) = e(\beta_1 \cup \beta_2 \cup \gamma) = \text{bind}(e, \beta_1 \cup \beta_2)(\gamma)$. A canvas binds every expression in its scene in this way. □

Proof of Proposition 35. From $r = \lceil n/c \rceil$, $rc \geq n > (r-1)c$, so the first $r-1$ rows are full and the last holds $n - (r-1)c \in [1, c]$ cells, leaving $rc - n < c$ empty. From $c = \lceil \sqrt{n} \rceil$, $n \leq c^2$, so $n/c \leq c$ and $r \leq c$. □

Change history

Release history of mosaickit. Documentation-only changes are not listed; each entry is tagged where the feature it describes is documented, following l3doc convention, so the page number is the real page.

v0.8.0

`DashStyle`: `DashStyle` and `ArrowStyle` accept short forms, so a package can use them as its own line and arrow style enums ····· 17

v0.5.1 (2026-10-03)

`MarkerLayer`: Markers are drawn whole on the plot edge; markers centred outside the plot are left out ····· 7

v0.5.0 (2026-10-03)

`ArrowLayer`: A dashed, dotted or dash-dot stroke dashes the shaft; the heads stay solid ····· 7

`CanvasGrid`: `links` added ····· 24

v0.4.0 (2026-10-03)

`BraceLayer`: Inside brace labels move out on a leader when nothing past the tip is free (they used to overlap and warn) ····· 8

v0.3.2 (2026-10-03)

`PointLabelLayer`: A region counts as the point's own only when the point is strictly inside it; a point on a region's edge keeps its label outside ··· 14

v0.3.1 (2026-10-03)

`PointLabelLayer`: A point label may sit inside a filled region that contains its point ····· 14

v0.3.0 (2026-10-03)

`AxisSpec`: Axis titles are placed past the arrow tips instead of centred on them ····· 8

`mosaickit`: Text outside an axis is laid out in columns: marks, then outside braces (one column per lane), then notes; marks and notes are spread so none overlap ····· 8

Point labels are placed before region labels, so region callouts avoid them; label text is measured with its rotation ····· 14

`AxisNoteLayer`: The default theme gains the `axes.note` role ····· 8

`Obstacles`: `Placement.leader` may be `None` ····· 15

`fits_inside`: `place_point_label` and `place_beside` added ····· 16

`Palette`: Themes and styles refer to colors by palette name, resolved against `Config.palette` when the render plan is built; an unknown name raises `ConfigurationError` naming the role, field and palette ····· 20

`Config`: `Config.palette` added, defaulting to `DEFAULT_PALETTE` ····· 21

`Config.load`: TOML accepts a `[palette]` table, and style colors may be palette names, checked when the file is loaded ····· 22

`SaveOptions`: `expand` added (default `True`) ····· 25

v0.2.0 (2026-10-03)

`mosaickit`: Built-in renderers are loaded by name on first use; the core no longer imports the Matplotlib backend ····· 3

`LayoutWarning`: Added, emitted when no callout position avoids every obstacle ····· 6

`PathLayer`: `clip` added (default `True`); arrowheads no longer redraw a solid shaft over the end of the path, so dashed arrows stay dashed ····· 7

`TextLayer`: `anchor` accepts the four corners `top-left`, `top-right`, `bottom-left`, `bottom-right` ··· 7

`ArrowLayer`: Arrowheads are no longer clipped to the axes ····· 7

`AxisSpec`: Axis presets draw filled triangle arrowheads; axis lines are not clipped, so they keep their full width on the plot edge ····· 8

`mosaickit.layout`: Added: the pure geometry behind label placement ····· 11

`DEFAULT_PALETTE`: Added: a grey ramp, `white`, `blue`, `red` and `teal`; the default theme takes its colors from it, so `primary`, `secondary` and `accent` become blue, red and teal ····· 20

`resolve`: `themes.resolve` accepts `overrides`, applied in order after the theme ····· 21

`Layer`: Layers declare `style_slots`; Matplotlib drawing is driven by a per-type registry ····· 25

v0.1.1 (2026-10-01)

General: Package metadata on PyPI links the homepage, repository, issue tracker, changelog and release notes ····· 2

v0.1.0 (2026-10-01)

General: First release: immutable scenes and layers, sparse styles, namespaced themes, canvases, spanning grids, parameter expressions, animation, Matplotlib rendering with job-scoped caches, and strict TOML configuration ····· 2

Command Index

A
Animation 24
ArrowLayer 7
ArrowPlacement 18
ArrowStyle 18
assign_lanes 10
AxisMarkLayer 8
AxisNoteLayer 8
AxisSpec 8
B
box_frame 8
Brace 13
brace_outline 13
BraceLayer 8
build_axes 8
C
CacheKey 26
Canvas 5
CanvasGrid 23
CanvasGrid.from_layout 24
CanvasGrid.sweep 24
CanvasSpec 4
Color 19
Config 21
Config.from_dict 22
Config.load 22
Constant 22
crosshair_axes 8
D
DashStyle 17
DEFAULT_PALETTE 19
distance_to_boundary 12
E
expand_roles 21
Expression 22
F
Fill 18
FillLayer 7
fits_inside 16
G
GridLink 24
GroupLayer 7
gutter_columns 10
I
Interval 5
L
Layer 6
Layout 24
LegendLayer 7
LegendStyle 18
M
Marker 18
MarkerLayer 7
O
Obstacles 15
P
Palette 19
Parameter 22
PathLayer 7
place_beside 16
place_callout 16
place_point_label 16
Placement 15
point_in_polygon 12
PointLabelLayer 14
polygon_edges 12
polylabel 13
Q
quadrant_axes 8
R
ray_exit 12
Rect 11
rect_hits_segment 11
rect_inside_polygon 12
rect_overlaps_polygon 12
RegionLabelLayer 14
register_builder 25
register_pass 25
RenderCache 26
Renderer 25
RendererRegistry 25
RolePack 21
S
SaveOptions 25
Scene 5
segments_intersect 11
Span 23
SpanBraceLayer 8
SparseStyle 17
spread 9
Stroke 17
StyleBundle 20
T
TextLayer 7
TextStyle 18
Theme 20
ThemeRegistry 21
TRANSPARENT 19
U
use_config 21

References

Agafonkin, V. (2016,). *A New Algorithm for Finding a Visual Center of a Polygon*. https://blog.mapbox.com/a-new-algorithm-for-finding-a-visual-center-of-a-polygon-7c77e6492fbc

Ayer, M., Brunk, H. D., Ewing, G. M., Reid, W. T., & Silverman, E. (1955). An Empirical Distribution Function for Sampling with Incomplete Information. *The Annals of Mathematical Statistics*, *26*(4), 641–647. https://doi.org/10.1214/aoms/1177728423

Barlow, R. E., Bartholomew, D. J., Bremner, J. M., & Brunk, H. D. (1972). *Statistical Inference under Order Restrictions*. Wiley.

Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

de Berg, M., Cheong, O., Kreveld, M. van, & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3rd ed.). Springer. https://doi.org/10.1007/978-3-540-77974-2

Haines, E. (1994). Point in Polygon Strategies. In P. S. Heckbert (Ed.), *Graphics Gems IV: Graphics Gems IV* (pp. 24–46). Academic Press.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55